DH2323 DGI Rendering Track

Lab 3: Rasterization

Processing (Java)

1 Introduction

Raytracing is the simplest way of computing a 2D image of a 3D scene. It can be used to simulate all kinds of phenomena, more than we implemented in the second lab. Its only disadvantage is the speed. Therefore, it is typically not used for real-time visualization. For those scenarios another algorithm called rasterization can be used. Although rasterization is typically faster than raytracing, it is not as easy to implement and it cannot be used to simulate all illumination phenomena. It is, for instance, very difficult to simulate multiple bounces of light with it. In this lab you will implement a rasterizer. The lab consists of three parts. In the first part you will explore:

- Perspective projection of 3D points by a pinhole camera
- Drawing 3D objects modeled by triangular surfaces by first projecting the vertices of the triangles to the 2D image plane.
- Drawing the edges of each triangle as lines in 2D, using linear interpolation.

The result of this can be seen in the top image in Figure 1. Then you'll add:

- Drawing of filled triangles.
- A depth buffer to check visibility for each pixel.

The result of this is very similar to the first part of Lab 2, as it can be seen in the middle image of Figure 1, but computed much faster. In the last part you will extend the program by also implementing:

- Light sources
- Interpolation of arbitrary quantities across the triangle, although you will focus on light.
- Per vertex and per pixel illumination, implemented by vertex and pixel shaders.

The final result can be seen in Figure 1.

Note: since we assume that you are somewhat familiar by this lab with Processing (Java), for some of the code examples below, you might need to implement some basic changes yourselves e.g. converting *PVector*s to *Color*s where appropriate, etc.

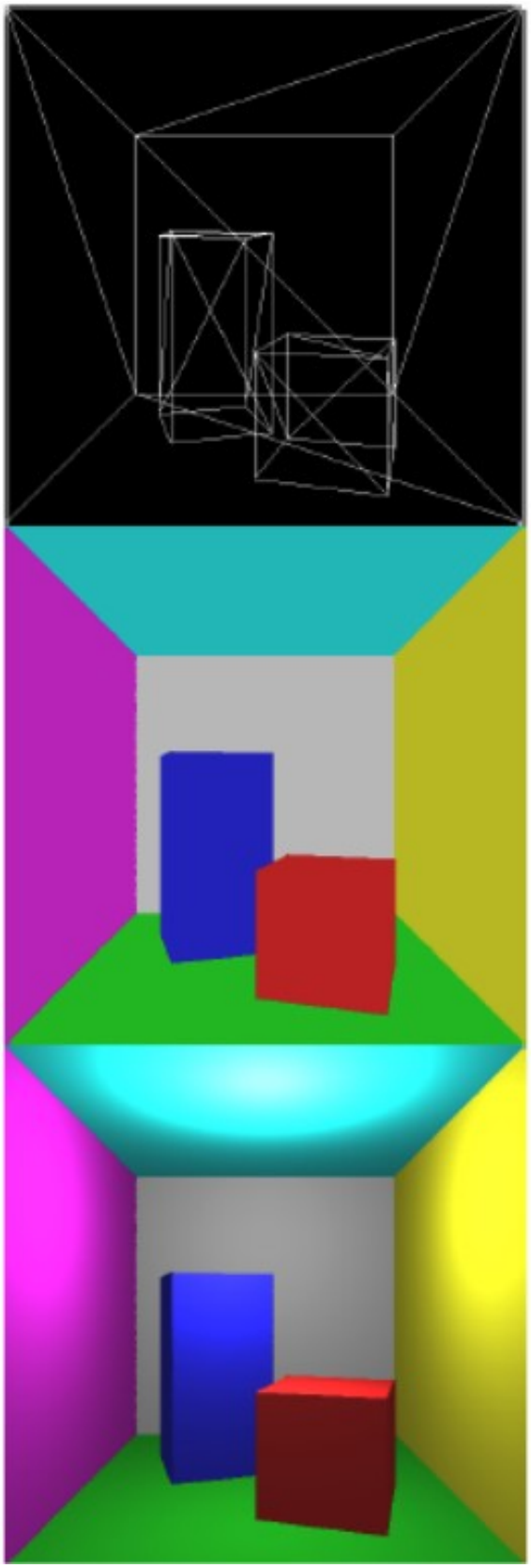

Figure 1: Expected final output for the lab

2 Transformations

Perspective Projection of Points from 3D to 2D

In the first lab you made a starfield by simulating a pinhole camera and its projection of 3D points to an image. In the second lab you did a raytracer by also simulating a pinhole camera, but you did not project points then. Instead you sent out rays for each pixel of the camera image. These are the two main approaches that can be used to make a 2D image of a 3D world.

In this lab we will go back to the first approach and use projection instead of raytracing. Assume we have a pinhole camera positioned at the origin looking along the z-axis of a 3D coordinate system. Let the x-axis point to the right and let the y-axis point downwards. Let f be the focal length of the camera, i.e. the distance from the camera position to the image plane, measured in pixel units. Let the x-axis and y-axis of the 2D camera image be the same as for the 3D world, i.e. the origin will be in the "middle" of the image. Then the projection of an arbitrary 3D point (X, Y, Z) to a 2D point (x, y) in the camera image can be written as:

$$x = f * \frac{X}{Z} \tag{1}$$

$$x = f * \frac{Y}{Z} \tag{2}$$

this relation can be derived by looking at two triangles that have the same angles. However, when working with computers it is common to have the origin of the image in the top left corner. If the image has width W and height H, the projection can then be written as:

$$x = f * \frac{X}{Z} + W2$$

$$x = f * \frac{Y}{Z} + H2$$

(3)

(4)

Translation

Assume the camera is not fixed at the origin but can move around freely. Let C be the position of the camera. Then to project a 3D point using equation 3 and 4 we first need to transform the point P from the world coordinate system to the system where the camera is at the origin. We can do this by subtracting the position of the camera:

$$P' = P - C \tag{5}$$

P' is then the position of the point in a coordinate system centered at the camera.

Rotation

Now we assume that the camera position is fixed at the origin again but that it can rotate around it. We then have a fixed world coordinate system and another coordinate system that rotates with the camera. The transformation of a position vector P from the world coordinate system to the system of the camera can then be expressed using a 3x3 matrix R and vector-matrix multiplication:

$$P'[1x3] = R[3x3]P[1x3] \tag{6}$$

where P and P' are row vectors. The matrix R that can be used to perform this transformation is the rotation matrix of the camera. Here we have used row vectors to represent 3D positions. We then multiply them on the left side of the matrix. It is of course also possible to use column vectors. Then the transformation from world system to camera system would have been:

$$P'[3x1] = R[3x3]P[1x3] \tag{7}$$

i.e. we would have multiplied the column vector on the right side of the matrix instead. Note that the rotation matrices are slightly different in those two cases, although they describe the same rotation. Each is the transpose of the other. There is no theoretical difference between the two approaches, but in practice it can be confusing, especially if they are mixed.

Translation and Rotation

Finally we assume that the camera can both rotate and translate. Then we can perform the transformation of a point from world coordinate system to camera coordinate system in two steps:

1. Transform it from world space to a coordinate system that is centered at the camera using equation 5.
2. Transform it from the coordinate system that is centered at the camera to a system that is also rotated as the camera using Equation 6.

The total transformation then becomes:

$$P' = (P - C)R \tag{8}$$

3 Drawing Points

Task 3.1 Write Draw function to project vertices of scene

First just try to project and plot the vertices of the scene, similar to how you did for the starfield in the first lab. The Draw function in the given skeleton program looks something like this:

```
void draw() {
    background(0);

    ArrayList<PVector> vertices = new ArrayList<PVector>();

    // Iterate over all triangles
    for (int i = 0; i < Globals.triangles.size(); i++) {
      Triangle triangle = Globals.triangles.get(i);
      vertices.clear();
      vertices.add(triangle.v0);
      vertices.add(triangle.v1);
      vertices.add(triangle.v2);

      PVector projPos = new PVector(0,0);
      for (int v=0; v<3; v++)
      {
        VertexShader (vertices.get(v), projPos);
        color c1 = color(1, 1, 1);
        set(int(projPos.x), int(projPos.y), c1);
      }
  }
  updatePixels();
}
```

Task 3.1 Implement VertexShader function

The function loops through all triangles and all vertices of the triangles and calls the function VertexShader on each. You have to implement the function:

```
void VertexShader (PVector v, PVector p);
```

It should take the 3D position of a vertex *v* and compute its 2D image position and store it in the given 2D integer PVector *p*. Here, we can again use a *PVector,* which also works for 2D integer vectors, i.e. in this case a pixel position will be represented by two integers. Thus it should handle the translation and rotation of the camera as well as the perspective projection.

Task 3.2 Implement rotation and translation in Update function and fix camera position

If you want, you can wait with implementing the rotation and also the motion of the camera in the Updatefunction. These things will be easier to debug later when you will see something more than just some sparse points on the screen. You can start by just having a fixed position of the camera represented by the global variable:

```
static PVector cameraPosition = new PVector(0,0,-3.001);
```

As you might remember from the second lab, our 3D scene consists of a cubic room placed at the origin and having a side length of 2. If we set the focal length, width and height of the camera to the same number and place the camera at (0, 0, -3.001), it will see the whole room. Why is that? Make sure that you understand this by drawing a Figure. If you got this working, you should see something similar to Figure 2, i.e. points at the corners/vertices of the room and the two boxes.

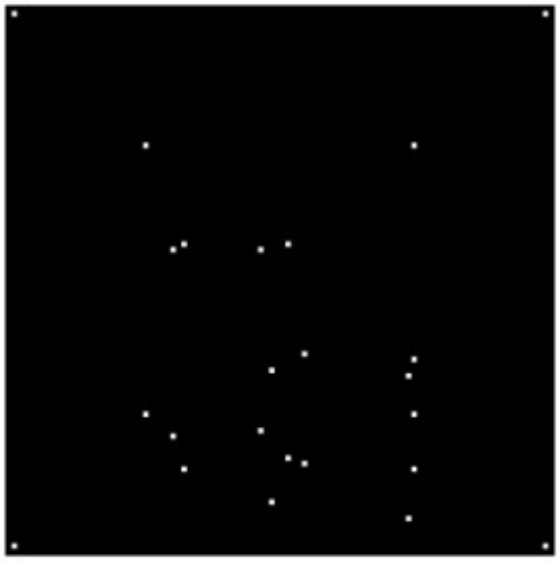

Figure 2: Drawing points projected by a pinhole camera.

4 Drawing Edges

To make the visualization slightly more interesting, we will try to also draw the edges of the triangles, instead of just the vertices. We can do this by drawing lines in 2D between the projected vertices of the triangle. To draw lines in 2D you can use a function that does linear interpolation similar to what you wrote for the first lab.

Task 4.1 Implement Interpolate function

Instead of interpolating pixel colors represented by *a PVector* (note: Processing also has a *color* class, but PVector works just fine), we will interpolate pixel positions represented by the 2D PVector. In this lab you will later extend the function to interpolate lots of different values and it is therefore convenient if the interpolation is implemented in a simple but efficient way. You might remember interpolation from T2.2 in Rendering Track Lab 1:

```
void Interpolate(PVector a, PVector b, int N, ArrayList<PVector> r){
  if (N == 1) {
    PVector midpoint = PVector.lerp(a, b, 0.5);
    r.add(midpoint);
    return;
  }
  for (int i = 0; i < N; i++) {
    float t = float(i) / (N-1);
    PVector interpolated = PVector.lerp(a, b, t);
    r.add(interpolated);
  }
}
```

Just as in the first lab, the Interpolation function takes the start value *a* and the end value *b* and fills the ArrayList of PVectors called *r* with *N* values linearly interpolated in between. For example, to create an ArrayList containing the values (4, 2), (3, 4), (2, 6), (1, 8) we can write:

```
ArrayList<PVector> result = new ArrayList<PVector>();
PVector a = new PVector(4,2);
PVector b = new PVector(1,8);
Interpolate(a, b, 4, result);
```

Make sure that you understand how the interpolation code works as you will use it a lot and also extend it to work for other quantities than 2D positions. It might be good to know that although this is a simple way to perform interpolation of integers it is not as fast as *Bresenham's line algorithm*.

Task 4.2 Declare variables for delta vector and number of pixels

However, that is less intuitive and nothing you need to worry about for this lab. Doing linear interpolation in this simple way is good enough for us. To draw a line in 2D we first need to know how many pixels the line should consist of. We do not want any holes in the line. Depending on whether the line is mostly horizontal or vertical we will use one pixel per column or one pixel per row. If *a* and *b* represent the start and end of the line segment we can then compute the number of pixels to draw as:

```
PVector delta = PVector.sub(a,b);
delta.x = abs(delta.x);
delta.y = abs(delta.y);
int npixels = int(max(delta.x, delta.y)) + 1;
```

You can then get the pixel positions of the line by calling the Interpolation function:

```
ArrayList<PVector> line = new ArrayList<Pvector>();
Interpolate(a, b, npixels, line);
```

Task 4.3 Write DrawLine function

When we have these we can loop through all of them and call *set()* for these pixel positions to draw the line. Write a function that does this:

```
void DrawLine(PVector a, PVector b, PVector col);
```

Note: for quick prototyping purposes, note that Processing has *line()* and *lerp()* functions that you can initially use. But you should implement your own for the final version.

Task 4.4 Write DrawPolygonEdges function

Before applying it to draw the edges of the projected triangles, try to draw lines between some arbitrary image points. When you got that working add another function to draw the edges of a triangle:

```
void DrawPolygonEdges(ArrayList<PVector> vertices)
{
  int V = vertices.size();
  // Transform each vertex from 3D world position to 2D image position
  ArrayList<PVector> projectedVertices = new ArrayList<PVector>(vertices);

  for (int i=0; i<V; ++i)
  {
    VertexShader(vertices.get(i), projectedVertices.get(i));
  }
  // Loop over all vertices and draw the edge from it to the next vertex
  for (int i =0; i<V; ++i)
  {
    int j = (i+1) % V; // The next vertex
    PVector col = new PVector(1,1,1);
    DrawLine(projectedVertices.get(i), projectedVertices.get(j), col);
  }
}
```

Task 4.5 Write DrawPolygonEdges function after computation of values

It takes the vertices of the triangle, or in fact any polygon, and project them to the image and then draw a line for each edge, using *DrawLine*. The % ('modulo') operator gives the remainder after division of two integers. We use it to make the next index after the last index wrap around to the first index. The operator works like this:

0%3 = 0
1%3 = 1
2%3 = 2
3%3 = 0
4%3 = 1
5%3 = 2
6%3 = 0 (9)

You can then use *DrawPolygonEdges* in the Draw-function like this:

```
void draw() {
    background(0);

    ArrayList<PVector> vertices = new ArrayList<PVector>();

    // Iterate over all triangles
    for (int i = 0; i < triangles.size(); i++) {
      Triangle triangle = triangles.get(i);
      vertices.clear();
      vertices.add(triangle.v0);
      vertices.add(triangle.v1);
      vertices.add(triangle.v2);

      DrawPolygonEdges(vertices);
  }
  updatePixels();
}
```

This should give you an image of a wire frame scene like in Figure 3.

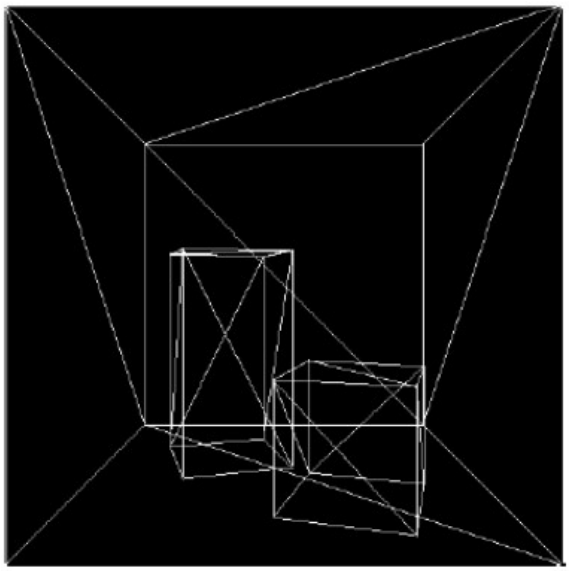

Figure 3: Drawing edges.

Task 4.6 Implement motion to the camera

Now, when you have a better visualization of the scene, you should also implement motion of the camera if you have not done that yet. There are global variables for the position of the camera and its rotation:

```
static PVector cameraPosition = new PVector(0,0,-3.001);
static Matrix3x3 R = new Matrix3x3();
static final float yaw = 0.0;
                    // Yaw angle controlling camera rotation around y-axis
```

Task 4.7 Update cameraPos and R matrix in Update function

Update these in the *Update()* function when the user presses the arrow keys just as you did in lab 2 and make sure that the VertexShader function handles both the translation and rotation of the camera before it projects a point from 3D to 2D. It should be possible to rotate the camera around the y-axis. This type of rotation is called yaw. If you want you can also add the possibility to rotate the camera up and down (pitch rotation), but that is not mandatory. Another thing that you can add if you want is control of the camera rotation by the mouse instead of the keyboard. This is also not mandatory. You can get access the relative motion of the mouse through the system variables *pmouseX* and *pmouseY*.

Similar system variables are used for other mouse details, for example the system variable *mouseButton* is set according to which mouse button is pressed. You can read more about it in the *Processing* reference documentation. When you move around with the camera you might notice that there are problems if the vertices are behind the camera. This is actually a bit tricky to fix and nothing you need to do for this lab. The solution to the problem is to perform *clipping* of the triangles before they are drawn. Clipping is a topic that could be studied in the project.

5 Filled Triangles

By just drawing the edges of the triangles we get some feeling for the 3D structure of the scene which might be good for some engineering visualizations. Nevertheless, it would be nice if we could also draw solid surfaces, i.e. fill the triangles with color. This turns out to be a bit more involved.

Declare left and right pixel's position

The main idea is to draw the triangle row by row. We have an array that stores the start position and another array that stores the end position of the triangle for each row:

```
ArrayList<PVector> leftPixels = new ArrayList<PVector>();
ArrayList<PVector> rightPixels = new ArrayList<PVector>();
```

Assume we have somehow filled these arrays with values, then it is simple to loop through them and draw each row from the start to the end. The tricky part is to compute the arrays in the first place.

As an example consider a triangle which has the projected vertices: (10,20), (30,10), (20,40), it should be drawn as 40-10+1=31 rows. The arrays for the left and right positions should then have 31 elements each. Corresponding elements should have the same y-coordinate but different x-coordinates: the left and right of that row.

One way to fill these arrays with values representing the polygon is to first initialize the start arrays with really big values and the end array with really small values:

```
for (int i=0 ; i<ROWS; ++i)
{
   leftPixels.get(i).x = Integer.MAX_VALUE;
   rightPixels.get(i).x = Integer.MIN_VALUE;
}
```

Task 5.1 Write logic for ComputePolygonRows function

We then loop through the edges of the polygon and compute the pixels corresponding to its line. Then for each y-coordinate of this line we check the corresponding elements in the left and right arrays. If the current x-value in the left array at that place is larger than the x-value of the line we replace it. If the current x-value in the right array at that place is smaller than the x-value of the line we replace it. After we have looped through all edges we then have

the smallest x-value for each row in the left array and the largest value for each row in the right array. Write a function that does this:

```
void ComputePolygonRows(ArrayList<PVector> vertexPixels, ArrayList<PVector>
leftPixels, ArrayList<PVector> rightPixels)
{
  //1. Find max and min y-value of the polygon and compute the
  //number of rows it occupies.

  //2. Resize leftPixels and rightPixels so that they have an
  //element for each row.

  //3. Initialize the x-coordinates in leftPixels to some really large
  //value and the x-coordinates in rightPixels to some really small value.

  //4. Loop through all edges of the polygon and use linear interpolation
   //to find the x-coordinate for each row it occupies. Update the corresponding
  //values in rightPixels and leftPixels.
}
```

It should take a vector with the projected position of the three vertices and compute the start and end image position of each row of the triangle. In fact, we can use this function not only to draw triangles but any convex polygon. You can test that your function produces sensible output by writing something like:

```
ArrayList<PVector> vertexPixels = new ArrayList<PVector>(3);
PVector p1 = new PVector(10, 5);
vertexPixels.set(0, p1);
PVector p2 = new PVector(5, 10);
vertexPixels.set(1, p2);
PVector p3 = new PVector(15, 15);
vertexPixels.set(2, p3);

ComputePolygonRows(vertexPixels, leftPixels, rightPixels);
for (int row=0; row<leftPixels.size(); ++row)
{
  //print debug info about leftPixels and rightPixels
}
```

This should give output similar to:

```
Start:    (10, 5). End: (10, 5).
Start:    (9, 6). End: (10, 6).
Start:    (8, 7). End: (11, 7).
Start:    (7, 8). End: (11, 8).
Start:    (6, 9). End: (12, 9).
Start:    (5, 10). End: (12, 10).
Start:    (7, 11). End: (13, 11).
Start:    (9, 12). End: (13, 12).
Start:    (11, 13). End: (14, 13).
Start:    (13, 14). End: (14, 14).
Start:    (15, 15). End: (15, 15).
```

Task 5.2 Draw computed rows of the triangles

```
void DrawRows(ArrayList<PVector> leftPixels, ArrayList<PVector> rightPixels)
```

This function should call *set()* for each pixel between the start and end for each row.

Task 5.3 Write function DrawPolygon and call it inside Draw function

Finally write a function that projects the vertices and calls the other two functions:

```
void DrawPolygon(ArrayList<PVector> vertices)
{
  int V = vertices.size();
  ArrayList<PVector> vertexPixels = new ArrayList<PVector>(V);
  for (int i=0; i<V; ++i)
    VertexShader(vertices.get(i), vertexPixels.get(i));

  ArrayList<PVector> leftPixels = new ArrayList<PVector>();
  ArrayList<PVector> rightPixels = new ArrayList<PVector>();
  ComputePolygonRows(vertexPixels, leftPixels, rightPixels);
  DrawPolygonRows(leftPixels, rightPixels);
}
```

Then call DrawPolygon in the Draw-function instead of DrawPolygonEdges. To signal what color to draw you can create a new global variable:

```
static PVector currentColor;
```

which you use in DrawPolygonRows when you call *set()*. We set this color to the color of the current triangle when we loop over all triangles and call DrawPolygon in the Draw-function:

```
void draw() {
    background(0);
    ArrayList<PVector> vertices = new ArrayList<PVector>();
    // Iterate over all triangles
    for (int i = 0; i < triangles.size(); i++) {
      Triangle triangle = triangles.get(i);
      currentColor = triangle.col;
      vertices.clear();
      vertices.add(triangle.v0);
      vertices.add(triangle.v1);
      vertices.add(triangle.v2);
      DrawPolygon(vertices);
    }
    updatePixels();
}
```

You should then get the result seen in Figure 4. When you manage to get that you are done with most of the coding for this lab. The remaining stuff does not require as much coding but greatly improves the image. Notice how the blue box is drawn on top of the red since we have not added any mechanism to deal with occlusion yet. Compare the speed of the program with the raytracer. **Question:** How much faster is it for this scene?

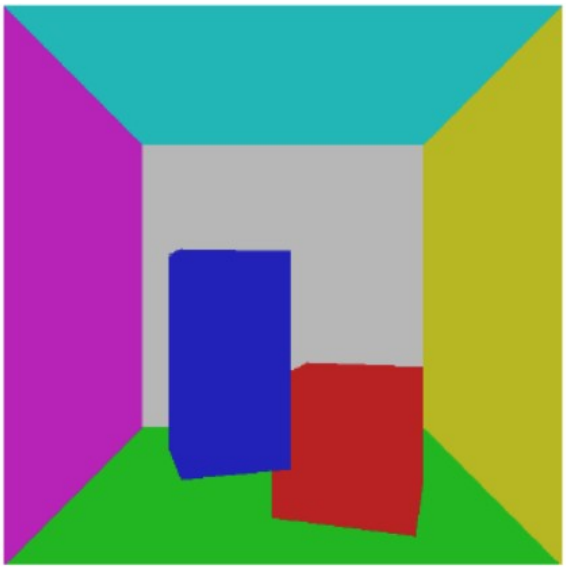

Figure 4: Filled triangles. Notice how the blue box is drawn on top of the red.

6 Depth Buffer

You have now implemented the core of a rasterizer. However, a problem with the current implementation is that the triangles might be drawn on top of each other, in arbitrary order. If multiple triangles occupy the same pixel we would like the one closest to the camera to be visible. In the raytracer we managed this by treating all pixels independently. For each pixel we followed a ray and looked at the closest intersection for that ray.

In a rasterizer we try to speed things up by not treating every pixel independently. We just treat every triangle independently. The standard way to handle this occlusion problem in a rasterizer is to use a so called depth buffer. That is an image storing a depth value for each pixel instead of a color value. For each pixel we thus store the depth of the closest surface point that has been drawn at that position so far. When we draw a new pixel we can then check if its depth is smaller than the one currently stored in the depth buffer. Only then do we draw it and also replace the value in the depth buffer.

To do this we need to keep track of the depth of each pixel in the image. We can do this by interpolating the depth over the triangle when we draw it. First we compute the depth of the vertices, in the coordinate system of the camera. Then we interpolate the depth over each edge of the triangle. Finally, we interpolate the depth over each row when we draw. This is similar to the bilinear interpolation we used in the first lab. First we interpolate the left and right edge from top to bottom. Secondly, we interpolate each row from left to right.

However, the perspective projection makes the interpolation a bit more involved. Assume z1 is the depth of one of the vertices and z2 is the depth of another vertex. Then in the 3D world the depth along the edge will vary linearly between z1 and z2, since the surface is planar. However, in the image the depth along the edge will not vary linearly! Instead it turns out that 1=z will vary linearly, due to the perspective projection of the camera. We can thus compute this quantity for each vertex and then interpolate it linearly across the projected 2D triangle.

Task 6.1 Declare DepthBuffer variable

After interpolation we could take the inverse to get back to the depth z. However, we do not really need the depth z. The inverse 1=z which we already have is good enough. We can then store 1=z in the depth buffer for each pixel that is drawn. A new pixel should then be drawn if its 1=z is larger than the one already stored in the depth buffer. Then its depth z will be smaller and it is closer to the camera. To implement a depth buffer we create a global 2D array with the same size as the camera image:

```
float[][] depthBuffer = new float[SCREEN_HEIGHT][SCREEN_WIDTH];
```

Task 6.2 Declare class for Pixel

In it we will store the inverse depth 1=z for each pixel. Since we now also need to interpolate the depth over the triangle, not just 2D position, we create a new class to hold the information needed for each pixel:

```
class Pixel {
  int x;
  int y;
  float zinv;
}
```

Task 6.3 Implement Interpolate function

Previously we just interpolated the position between the vertices when we a triangle. Now we also want to interpolate the inverse depth. Thus, you need to implement an Interpolation function that interpolates our *Pixel* class linearly instead of the 2D PVector:

```
void Interpolate(Pixel a, Pixel b, int N, ArrayList<PVector> r)
```

Task 6.4 Change some other functions too

After you have done this you also need to change *ComputePoloygonRows*, *DrawPolygonRows*, *VertexShader* and *DrawPolygon* to handle *Pixel* instead of *PVector*:

```
void ComputePolygonRows(ArrayList<Pixel> vertexPixels, ArrayList<Pixel>
leftPixels, ArrayList<Pixel> rightPixels)

void DrawPolygonRows(ArrayList<Pixel> leftPixels, ArrayList<Pixel> rightPixels)

void VertexShader(PVector v, Pixel p)
```

Thus, in *VertexShader*, you should now compute the inverse depth *zinv* for the vertex in the coordinate system of the camera, before you compute its projected image position (*x,y*).

If you do all of the above, you will have access to *zinv* for every pixel that you draw in *DrawPolygonRows*. Before drawing a new pixel, you can then check if it is in front of the one that is currently drawn there by comparing with the value of depthBuffer at that pixel. If it is in front you can draw it and update the value in the depth buffer. Remember that you also need to clear the depth buffer in the beginning of the Draw-function before you start drawing. You do this by setting all pixels to represent infinite depths, i.e. zero inverse depths:

```
for (int y=0; y<Globals.SCREEN_HEIGHT; ++y)
    for (int x=0; x<Globals.SCREEN_WIDTH; ++x)
      depthBuffer[y][x] = 0;
```

You should then get surfaces that correctly occlude each other like in Figure 5.

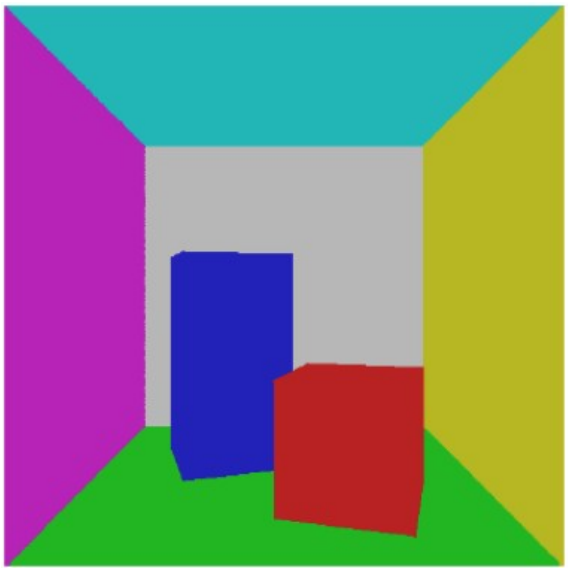

Figure 5: Filled triangles. Notice how the blue box is no longer drawn on top of the red.

7 Illumination

Your rasterizer now renders the same image as the first version of the ray-tracer, but hopefully much faster. We will now continue and also add illumination. When computing the illumination in a rasterizer we have two choices. We could do the computations for every vertex and then just interpolate the result or we could do the computations for every pixel. The first approach will be faster but less accurate.

Task 7.1 Create functions VertexShader and PixelShader

Either way it is convenient to have a function that handles whatever computations we want to do per vertex:

```
void VertexShader(PVector v, Pixel p)
```

which we already have, and another function which handles whatever computations we want to do per pixel:

```
void PixelShader (Pixel p)
```

This function should be called for every interpolated pixel in *DrawPolygonRows* and handle the drawing. In the current version of our razterizer it might look like:

```
void PixelShader(Pixel p)
{
  float[][] depthBuffer = new float[Globals.SCREEN_HEIGHT][Globals.SCREEN_WIDTH];

  int x = p.x;
  int y = p.y;
  if (p.zinv > depthBuffer[y][x])
  {
    depthBuffer[y][x] = p.zinv;
    set(x,y,currentColor);
  }
}
```

Task 7.2 Modify VertexShader function

Currently VertexShader takes a 3D PVector and computes a Pixel. You have probably written something like:

```
void VertexShader(PVector v, Pixel p)
{
  PVector pos = PVector.sub(v, cameraPosition);
  R.multiply(pos);
  p.zinv = 1/pos.z;
  p.x = int (focalLength * pos.x * p.zinv) + SCREEN_WIDTH/2;
  p.y = int (focalLength * pos.y * p.zinv) + SCREEN_HEIGHT/2;
}
```

Task 7.3 Create new class Vertex

To make our rasterizer a bit more general and abstract we can create a new type which describes a general vertex:

```
class Vertex
{
  PVector position;
};
```

For now we just store the position of the vertex since that is all we use at the moment. However, in general we could also assign many other quantities to the vertex. We will soon do this to handle illumination but first make your program handle this simple Vertex-type by updating: *Draw*, *DrawPolygon* and *VertexShader*.

The flow of your general rasterizer can now be described as:

- VertexShader is called for each Vertex and computes a Pixel.
- These Pixels are interpolated in the image between the vertices. First vertically along the edges and then horizontally across the polygon.

- Each such interpolated Pixel is sent to PixelShader which determines the final color of the image at that position.

Most of the code you have written is to perform the second step. There is not that much code in VertexShader and PixelShader. The good news is that to render more sophisticated images, e.g. with illumination and texture mapping, you do not need to add things to the second step. You just need to alter VertexShader, PixelShader and the interpolation function.

Per Vertex Illumination

We will first try to implement the **illumination computations for every vertex** and then interpolate these values across the polygon, similar to how we interpolated zinv. In lab 2 you learned that the direct illumination from an omni light source to a surface point can be written as:

$$D = (P * max(r.\hat{} n, \hat{} 0))/4\pi r^2 \tag{9}$$

where D is the power of the incoming direct light per surface area. P is the power of the light source, r is a vector from the surface point to the light source and $\hat{}n$ is the normal of the surface. For ideally diffuse surfaces with reflectance ρ the total reflected light is then:

$$R = \rho * (D + N) \tag{10}$$

where N is the incoming indirect illumination reflected from other surfaces. We approximated N as a constant term.

Task 7.4 Define light position, power and area of indirectLightPower

To implement this illumination model we will store the parameters of the light source as global variables just as in Lab 2:

```
static PVector lightPos = new PVector(0,-0.5, -0.7);
static PVector lightPower = new PVector(1.1,1.1,1.1);
static PVector indirectLightPowerPerArea = new PVector(0.5,0.5,0.5);
```

We would initially like to compute the illumination for every vertex in VertexShader. We then need to evaluate equation 9 and 10 for every vertex.

Task 7.5 Redefine Vertex and Pixel classes

Besides the global light variables we then need to know the position, normal and reflectance at the vertex. It is therefore convenient to add this information to our Vertex class:

```
class Vertex
{
  PVector position;
  PVector normal;
  PVector reflectance;
};
```

Make sure to set this information for every vertex in the Draw function. After you have done this you will have access to this in VertexShader. You can then compute the illumination, but you also need to store it for the resulting output Pixel of VertexShader. You therefore need to add another quantity to your Pixel class to store this:

```
class Pixel {
  int x;
  int y;
  float zinv;
  PVector illumination;
};
```

Task 7.6 Interpolate values by modifying PixelShader function

Now the illumination gets computed for every Vertex in VertexShader. We then want to interpolate this value over the polygon before we use it in PixelShader. To do this you need to modify the Interpolation-function so that it also interpolates the illumination quantity in Pixel. After you have done this you can simply access the illumination in PixelShader and use it when you draw the pixel:

```
void PixelShader(Pixel p)
{
  int x = p.x;
  int y = p.y;
  if (p.zinv > depthBuffer[y][x])
  {
    depthBuffer[y][x] = p.zinv;
    set(x,y, p.illumination);
  }
}
```

This should give you the result seen in Figure 6. As you can see the result is similar to the raytracer with illumination but not exactly the same. Since we are interpolating the illumination over the surfaces we get less detail, but gain speed.

Per Pixel Illumination

To get a more detailed illumination we will now compute unique values for every pixel. We do this by evaluating equation 9 and 10 in PixelShader instead of VertexShader.

Task 7.7 Define normal and reflectance vectors

We then need to have access to the 3D position that the pixel represents as well as its normal and reflectance. Since we model the normal and reflectance as constant over each triangle we do not need to interpolate them. Instead we can store these values for the triangle that is currently drawn in global variables:

```
PVector currentNormal;
PVector currentReflectance;
```

which we set in the draw function before drawing a triangle.

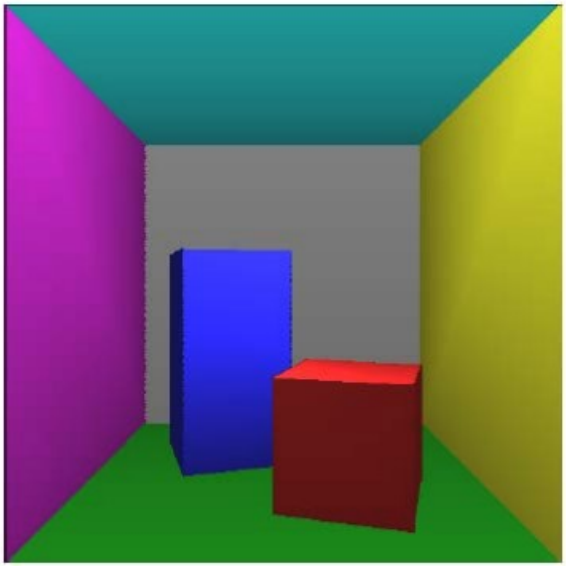

Figure 6: Illumination computed for vertices and interpolated in between.

Task 7.8 Add pixel positions to the Pixel class

However, the 3D position that the pixel corresponds to will be different for different pixels of the triangle and we thus have to interpolate it. You therefore need to add it to the Pixel class:

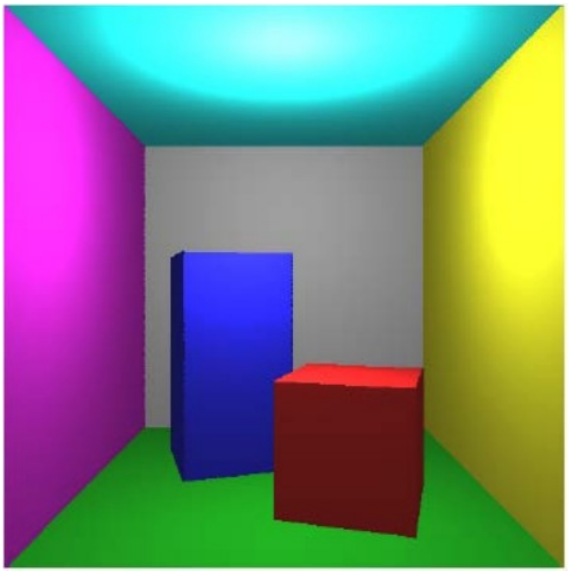

Figure 7: Unique illumination computed for every pixel.

```
class Pixel {
  int x;
  int y;
  float zinv;
  PVector pos3d;
};
```

Task 7.9 Remove variables from Vertex class, interpolate values in Interpolation

Since we will no longer interpolate the illumination we do not need a variable for it in the Pixel class. Also since we will not compute the illumination in the VertexShader you can remove the normal and reflectance from the Vertex class, since they will not be used:

```
class Vertex
{
  PVector position;
};
```

Instead of computing the illumination in VertexShader you just need to store the 3D position of the Vertex to the corresponding variable in Pixel. Then you should interpolate this value in Interpolation instead of illumination, but in the same way though. You will then have access to the 3D position of the pixel in PixelShader and you can compute the illumination. You should then get the image seen in Figure 7. However, it is likely that you will instead get the result seen in Figure 8, where the illumination looks a bit skew. This happens if you do not implement perspective correct interpolation, but just interpolate the 3D position linearly. As long as the light source does not move this does not look too bad, but if the light is moving it does not look so good.

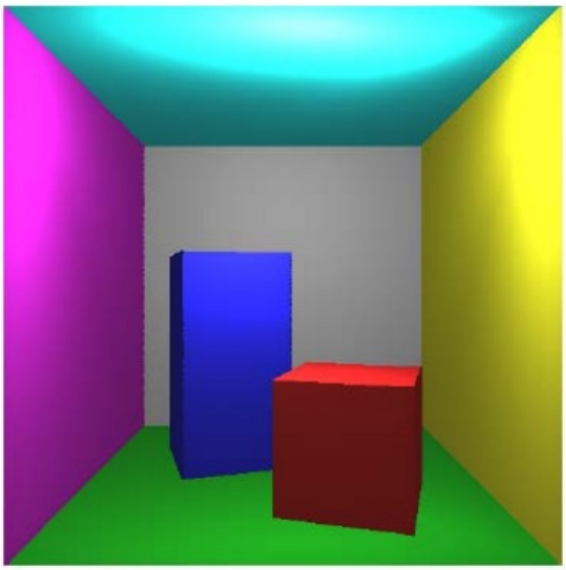

Figure 8: Unique illumination computed for every pixel, but not using perspective correct interpolation for the 3D position of the pixel.

Last details to finish the lab

When you interpolated the depth z over the image you could not interpolate z linearly. Instead you had to us the quantity 1/z to have something that changes linearly in the image. In general if you have some quantity q that changes linearly in the 3D world, it will not change linearly in an image created by perspective projection. Instead q/z will change linearly in the image. For instance, if we want to interpolate the 3D position p we can interpolate p/z linearly, and then after interpolation we can multiply it with z, or equivalently divide with 1/z, to get the interpolated 3D position that we want. Make sure that you do this when you interpolate pixels.

Finally, make sure that you can move the light source using the *w,a,s,d* keys as in the second lab and you are done with this lab as well. Your rasterizer should then render the same image as your final raytracer except for shadows. This is one of the things that are more difficult to compute in a rasterizer.

8 Extending to a Project

If you would like to work more with your software rasterizer for the project, these are things that could potentially be added:

- Shadows
- Loading of general models
- Textures
- Normal mapping

You could also work with the hardware rasterization libraries OpenGL, Vulkan and Microsoft's DirectX. They allow you to do rasterization using the GPU of the graphics card, which is designed for this special purpose, rather than using the more general but slower CPU, which we have done so far. This should give a significant speed up (e.g. by a factor of 100 or more).